

UNIVERSIDAD ... - FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA DE GESTIÓN DE MOVILIDAD PARA LA EMPRESA AVENTURA VIP DE CUSCO

TESIS PARA OPTAR EL TÍTULO PROFESIONAL DE INGENIERO DE SISTEMAS

Autor: (NOMBRES Y APELLIDOS DEL TESISISTA)

Asesor: (NOMBRES Y APELLIDOS DEL ASESOR)

CUSCO - PERÚ

2026

UNIVERSIDAD ... - FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

(COMPLETAR CON LOS DATOS DE LA UNIVERSIDAD)

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA DE GESTIÓN DE MOVILIDAD PARA LA EMPRESA AVENTURA VIP DE CUSCO

TESIS PARA OPTAR EL TÍTULO PROFESIONAL DE
INGENIERO DE SISTEMAS

Autor: (NOMBRES Y APELLIDOS DEL TESISISTA)

Asesor: (NOMBRES Y APELLIDOS DEL ASESOR)

CUSCO - PERÚ

2026

TABLA DE CONTENIDOS

- 1. INTRODUCCIÓN**
- 2. CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA**

1. Descripción de la realidad problemática
2. Formulación del problema
3. Objetivos de la investigación
4. Justificación e importancia
5. Alcance y limitaciones
3. **CAPÍTULO II: MARCO TEÓRICO**
 1. Antecedentes de la investigación
 2. Bases teóricas
 3. Marco conceptual
4. **CAPÍTULO III: METODOLOGÍA**
 1. Tipo y diseño de investigación
 2. Metodología de desarrollo de software
 3. Técnicas e instrumentos de recolección de datos
 4. Población y muestra
5. **CAPÍTULO IV: TECNOLOGÍAS Y HERRAMIENTAS**
 1. Stack tecnológico
 2. Justificación de las tecnologías elegidas
6. **CAPÍTULO V: ARQUITECTURA Y DISEÑO DEL SISTEMA**
 1. Arquitectura de software propuesta
 2. Diseño de la base de datos
 3. Diseño del módulo de autenticación
 4. Modelo de seguridad (roles y permisos)
7. **CAPÍTULO VI: RESULTADOS Y ESTADO DEL DESARROLLO**
8. **CONCLUSIONES**
9. **RECOMENDACIONES**
10. **REFERENCIAS BIBLIOGRÁFICAS**

RESUMEN

El presente trabajo de investigación tiene como objetivo diseñar e implementar un **sistema de gestión de movilidad** para la empresa **Aventura Vip de Cusco**, dedicada al transporte de pasajeros y que presta servicios corporativos al grupo **Belmond** (Hoteles Monastery y Palacio Nazarenas) mediante vales de viaje y servicios externos en efectivo.

La investigación surge de la necesidad de resolver problemas operativos concretos: errores en las reservas realizadas por WhatsApp y teléfono, vales de papel que se pierden o se malogran, documentos de la flota desorganizados, ausencia de control sobre los conductores, una cola nocturna de despacho que no se respeta y servicios en efectivo sin control financiero.

Como solución se propone un sistema multiplataforma construido sobre una **arquitectura multicapa con lógica de negocio centrada en la base de datos** (SP-Centric) y un **backend monolito modular** en Node.js con Express y TypeScript, una base de datos **PostgreSQL 15** y un frontend en **React**. La seguridad se garantiza mediante tokens **JWT** y un modelo de control de acceso basado en permisos (PBAC) con claves cifradas con **bcrypt**.

Al cierre del presente informe se encuentran implementadas y funcionales las **Fases 1 y 2** del sistema: el **módulo de autenticación** (login con reglas por tipo de cuenta y rate limit), el **dashboard integral** responsive con menú lateral y modo oscuro/claro, el **módulo de gestión de usuarios** (crear, listar, editar, permisos por checkboxes y accesos por rol) y el **módulo de organización** (empresas, establecimientos y áreas). Todo validado con **35 pruebas de integración automatizadas** que pasan contra la base de datos real. Los resultados permiten afirmar que la arquitectura propuesta (100% SP-Centric) es viable, segura y escalable para las fases posteriores del sistema.

INTRODUCCIÓN

La transformación digital del transporte urbano ha dado paso a plataformas de movilidad que gestionan flotas, conductores, reservas y pagos en tiempo real. Sin embargo, muchas empresas tradicionales de taxis -como Aventura Vip de Cusco- aún operan con procesos manuales y desorganizados que generan pérdidas

económicas, errores operativos y descontento en los clientes corporativos.

La presente investigación aborda esta problemática mediante el diseño e implementación de un sistema de gestión de movilidad a la medida de la empresa. El trabajo se estructura de la siguiente manera: el Capítulo I plantea el problema y define los objetivos; el Capítulo II desarrolla el marco teórico; el Capítulo III describe la metodología; el Capítulo IV presenta las tecnologías utilizadas; el Capítulo V detalla la arquitectura y el diseño del sistema; y el Capítulo VI muestra el estado actual del desarrollo. Finalmente se presentan las conclusiones y recomendaciones.

CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA

1.1 Descripción de la realidad problemática

Aventura Vip de Cusco es una empresa dedicada al transporte de pasajeros en la ciela de Cusco. Su principal cliente es el grupo **Belmond**, para el cual brinda servicio de traslados a los huéspedes de los hoteles **Monastery** y **Palacio Nazarenas**. El servicio corporativo se factura mediante **vales de viaje** que Belmond paga mensualmente según la distancia recorrida. Además, la empresa atiende servicios externos cobrados en efectivo.

Tras el análisis del proceso operativo, se identificaron los siguientes problemas críticos:

N°	Problema identificado	Consecuencia
1	Las reservas se solicitan por WhatsApp y teléfono	Errores de tipeo en nombres, destinos y horarios; pérdida de información.
2	Los vales de viaje se emiten en papel	Se pierden o se dañan; dificulta la conciliación mensual con Belmond.
3	Los documentos de la flota están desorganizados	SOAT, revisiones técnicas y seguros sin control de vigencia.
4	No existe gestión de conductores	Desconocimiento de disponibilidad, horarios y desempeño.
5	La cola nocturna (despacho a las 10:30 pm) no se respeta	Conflictos entre conductores y despacho injusto.
6	Los servicios en efectivo no se controlan	Pérdidas financieras y falta de registro de ingresos.

1.2 Formulación del problema

Problema general

¿De qué manera el diseño e implementación de un sistema de gestión de movilidad mejora el control operativo, la seguridad y la gestión financiera de la empresa Aventura Vip de Cusco?

Problemas específicos

1. ¿Cómo reducir los errores en el registro de reservas de servicio?
2. ¿Cómo digitalizar el control de los vales de viaje de Belmond?
3. ¿Cómo organizar la documentación de la flota y la gestión de conductores?
4. ¿Cómo garantizar el respeto del orden de despacho en la cola nocturna?
5. ¿Cómo asegurar que solo personal autorizado acceda al sistema?

1.3 Objetivos de la investigación

Objetivo general

Diseñar e implementar un sistema de gestión de movilidad multiplataforma para la empresa Aventura Vip de Cusco, que permita automatizar el registro de reservas, el control de vales, la gestión de flota y conductores, el despacho de la cola nocturna y el control financiero, garantizando la seguridad de la información.

Objetivos específicos

1. Implementar un módulo de autenticación seguro (login) basado en tokens JWT y claves cifradas con bcrypt.
2. Diseñar una base de datos relacional normalizada que soporte usuarios, roles, permisos, empresas y áreas.
3. Implementar un modelo de control de acceso basado en permisos dinámicos (PBAC) que permita administradores con acceso total o parcial.
4. Diseñar una arquitectura escalable que permita incorporar módulos de reservas, flota, conductores, cola nocturna y finanzas.
5. Validar el funcionamiento del sistema mediante pruebas de extremo a extremo (base de datos, API y consumo REST).

1.4 Justificación e importancia

La investigación se justifica desde tres perspectivas:

- **Justificación práctica:** la empresa necesita resolver problemas operativos que generan pérdidas económicas y afectan su relación con el cliente Belmond. El sistema automatiza los procesos manuales y reduce errores.
- **Justificación tecnológica:** se aplican tecnologías modernas y robustas (PostgreSQL, Node.js, React) sobre una arquitectura con lógica de negocio en base de datos, lo que garantiza integridad, rendimiento y seguridad.
- **Justificación académica:** el proyecto integra conocimientos de ingeniería de software, bases de datos, arquitectura de software, seguridad informática y desarrollo web, demostrando competencias profesionales.

1.5 Alcance y limitaciones

El alcance del presente trabajo cubre el análisis, diseño e implementación del módulo de autenticación, junto con el diseño completo del esquema de base de datos y la arquitectura del sistema. Las fases de reservas, flota, conductores, cola nocturna y finanzas forman parte del diseño y se desarrollarán en etapas posteriores.

Las principales limitaciones son la disponibilidad de tiempo para el desarrollo de todos los módulos y el acceso limitado a la información financiera detallada de la empresa por políticas de confidencialidad.

CAPÍTULO II: MARCO TEÓRICO

2.1 Antecedentes de la investigación

Existen diversos trabajos relacionados con sistemas de gestión de transporte y movilidad. Plataformas como Uber, Cabify y DiDi han demostrado la viabilidad de los sistemas de despacho de vehículos basados en aplicaciones móviles. Sin embargo, la mayoría de estas plataformas están orientadas al mercado de consumo masivo y no cubren las necesidades específicas de las empresas de taxi tradicionales que atienden clientes corporativos mediante vales y crédito mensual, como es el caso de Aventura Vip de Cusco.

En el ámbito local, la mayoría de empresas de transporte de Cusco operan con herramientas genéricas (hojas de cálculo y mensajería instantánea), lo que evidencia la necesidad de soluciones a medida que integren reservas, flota, conductores y finanzas en una sola plataforma.

2.2 Bases teóricas

2.2.1 Patrón Modelo-Vista-Controlador (MVC)

El patrón MVC (Model-View-Controller) es un estilo de arquitectura de software que separa la aplicación en tres componentes lógicos:

- **Modelo (Model):** gestiona los datos, el estado y las reglas de negocio. En el presente proyecto corresponde a la base de datos PostgreSQL, incluyendo las tablas y los procedimientos almacenados (SP_LOGIN, etc.).
- **Vista (View):** es la interfaz de usuario. Corresponde al frontend en React, que presenta los datos y captura la interacción del usuario.
- **Controlador (Controller):** actúa como intermediario entre la Vista y el Modelo. Recibe las peticiones HTTP, invoca los procedimientos de la base de datos y devuelve las respuestas. En el proyecto corresponde al servidor Express con TypeScript.

La separación de responsabilidades del patrón MVC permite que la Vista pueda cambiarse (por ejemplo, pasar de una aplicación web a una app móvil) sin modificar el Modelo ni el Controlador.

2.2.2 Arquitectura multicapa

La arquitectura multicapa organiza el sistema en capas con responsabilidades claramente definidas:

- **Capa de presentación:** la interfaz de usuario (React).
- **Capa de aplicación o transporte:** la API REST (Node.js + Express).
- **Capa de datos:** la base de datos (PostgreSQL 15).

2.2.3 Arquitectura con lógica de negocio en base de datos (SP-Centric)

En esta arquitectura, la lógica de negocio vive en **procedimientos almacenados** (stored procedures) dentro de la base de datos. La API se convierte en una "capa fina" que únicamente recibe peticiones HTTP, llama a los procedimientos y devuelve los resultados. Sus ventajas son:

- **Rendimiento:** la lógica se ejecuta cerca de los datos, reduciendo la latencia de red.
- **Integridad:** las transacciones y validaciones se ejecutan de forma atómica en la base de datos.
- **Seguridad:** la API no tiene permisos directos sobre las tablas, solo de ejecución sobre procedimientos autorizados, lo que mitiga la inyección SQL.
- **Consistencia:** al concentrar la lógica, cualquier cliente (web, móvil, API) obtiene el mismo comportamiento.

2.2.4 Monolito modular

Un monolito modular es una aplicación única desplegada como un solo servicio, pero organizada internamente en módulos independientes (auth, reservas, flota, conductores, finanzas). Combina la simplicidad de despliegue del monolito con la claridad y mantenibilidad de la modularidad.

2.2.5 Autenticación con tokens JWT

Los JSON Web Tokens (JWT) son tokens firmados digitalmente que permiten autenticar y autorizar peticiones sin almacenar sesiones en el servidor. El token contiene el identificador del usuario, su rol y sus permisos, y se firma con una clave secreta. El presente sistema emite tokens con una vigencia de 12 horas.

2.2.6 Control de acceso basado en permisos (PBAC)

A diferencia del control de acceso basado en roles (RBAC) tradicional, el modelo PBAC asigna **permisos específicos** a cada usuario. Esto permite que existan administradores con acceso parcial, y que el sistema crezca agregando nuevas filas de permisos sin modificar el código.

2.2.7 Cifrado de contraseñas con bcrypt

bcrypt es una función de hash de contraseñas basada en el algoritmo Blowfish, que incluye un factor de trabajo configurable ("salt") que dificulta los ataques de fuerza bruta. En el proyecto se usa a través de la extensión **pgcrypto** de PostgreSQL con la función `crypt()` y el generador de salt `gen_salt('bf', 8)`.

2.3 Marco conceptual

Término	Definición
API REST	Interfaz de programación basada en HTTP que sigue los principios de representación de estado.
Vale de viaje	Comprobante emitido por el servicio corporativo que se paga mensualmente según la distancia.
Cola nocturna	Orden de despacho de vehículos al cierre del servicio diario (10:30 pm).
Flota	Conjunto de vehículos de la empresa con su documentación (SOAT, revisiones).
Despacho	Asignación de un conductor a un servicio solicitado.

CAPÍTULO III: METODOLOGÍA

3.1 Tipo y diseño de investigación

La investigación es de tipo **aplicada**, ya que busca resolver un problema práctico de la empresa mediante la

aplicación de conocimientos tecnológicos. El enfoque es **cuantitativo** complementado con técnicas cualitativas de observación, y el diseño es **no experimental-transversal**, orientado al desarrollo e implementación de un producto tecnológico.

3.2 Metodología de desarrollo de software

Para el desarrollo del sistema se utiliza una metodología **ágil e incremental adaptada**, inspirada en Scrum y XP, dado que el proyecto avanza por fases funcionales:

Fase	Descripción	Estado
Fase 1 - Autenticación	Esquema de BD, login seguro, tokens JWT y control de permisos.	Completada
Fase 2 - Gestión de usuarios	CRUD de usuarios, empresas y áreas.	Pendiente
Fase 3 - Reservas	Registro de servicios y control de vales.	Pendiente
Fase 4 - Flota y conductores	Vehículos, documentos y disponibilidad.	Pendiente
Fase 5 - Cola nocturna y GPS	Despacho ordenado y ubicación de vehículos.	Pendiente
Fase 6 - Finanzas	Reportes de vales y servicios en efectivo.	Pendiente

3.3 Técnicas e instrumentos de recolección de datos

- **Observación directa:** se observaron los procesos operativos de la empresa (toma de reservas, despacho, manejo de vales).
- **Entrevistas:** entrevistas no estructuradas al personal administrativo y operativo.
- **Análisis documental:** revisión de vales de viaje, registros de la flota y reportes financieros.
- **Pruebas de software:** pruebas funcionales de la API REST mediante curl y verificación directa de los procedimientos almacenados.

3.4 Población y muestra

La población está conformada por el personal de Aventura Vip de Cusco: personal administrativo, operadores, conductores y usuarios del servicio. Para las pruebas se utilizó una muestra representativa que incluye cuentas de prueba de los cuatro roles del sistema (ADMIN, OPERADOR, CONDUCTOR, USUARIO).

CAPÍTULO IV: TECNOLOGÍAS Y HERRAMIENTAS

4.1 Stack tecnológico

Capa	Tecnología	Rol en el sistema
Base de datos	PostgreSQL 15 (Docker)	Almacenamiento, procedimientos almacenados (100% SP-Centric) y triggers de auditoría.
Backend	Node.js + Express + TypeScript	API REST, middleware de autenticación JWT, permisos (PBAC) y rate limit.
Frontend	React + Vite + TypeScript	SPA con dashboard integral: login, gestión de usuarios y organización.
Estilos	Tailwind CSS	Diseño responsive con la paleta oficial de la marca (negro, marrón #3D271C y ámbar #C17C26).
Iconografía	lucide-react	Iconos del menú lateral y de la interfaz.
Tipografía	Azonix + Caviar Dreams	Azonix (títulos/lujo) y Caviar Dreams (texto), servidas en WOFF2.
PWA	vite-plugin-pwa (Service Worker + manifest)	Aplicación instalable en el celular y funcional sin conexión.
Autenticación	JSON Web Tokens (JWT)	Emisión y validación de tokens de acceso (12 horas).

Capa	Tecnología	Rol en el sistema
Cifrado	bcrypt (pgcrypto)	Cifrado de contraseñas en la base de datos.
Seguridad HTTP	helmet + express-rate-limit	Cabeceras seguras y límite de intentos de login (anti fuerza bruta).
Pruebas	vitest + supertest	35 pruebas de integración automatizadas contra la BD real.
Contenedores	Docker	Despliegue del servidor de base de datos.

4.2 Justificación de las tecnologías elegidas

- **PostgreSQL 15:** sistema de gestión de bases de datos relacional de código abierto, robusto, con soporte completo de procedimientos almacenados (PL/pgSQL), triggers, JSONB y la extensión pgcrypto.
- **Node.js con Express:** plataforma asíncrona de alto rendimiento ideal para APIs REST. TypeScript añade tipado estático que mejora la calidad y el mantenimiento del código.
- **React:** biblioteca para construir interfaces de usuario modernas y escalables; combinado con Vite y Tailwind CSS permite una SPA rápida y adaptada a plataformas móviles.
- **PWA (Progressive Web App):** con vite-plugin-pwa la aplicación genera un manifest y un service worker, lo que la hace instalable en el celular y usable sin conexión, ideal para conductores en zonas con poca señal.
- **JWT:** estándar ampliamente aceptado para autenticación sin estado, ideal para arquitecturas REST y multiplataforma.
- **helmet y express-rate-limit:** el primero endurece las cabeceras HTTP (CSP, X-Frame-Options, HSTS); el segundo limita los intentos de inicio de sesión por IP (HTTP 429) para mitigar ataques de fuerza bruta.
- **vitest + supertest:** permiten ejecutar pruebas de integración contra la base de datos real y los procedimientos almacenados, garantizando que la lógica de negocio funciona antes de cada entrega.
- **Docker:** facilita el despliegue y la reproducibilidad del entorno, especialmente de la base de datos.

CAPÍTULO V: ARQUITECTURA Y DISEÑO DEL SISTEMA

5.1 Arquitectura de software propuesta

La arquitectura elegida es una **arquitectura multicapa con lógica de negocio centrada en la base de datos (SP-Centric)**, combinada con un **monolito modular** en el backend. El flujo de una petición es el siguiente:

Cliente (React / curl / app móvil)

| 1. POST /api/auth/login {correo_o_dni, clave}

v

Capa de aplicación - Servidor Node.js (Express + TypeScript)

| - Enrutamiento HTTP
| - Validación de formato
| - Middleware JWT y permisos

| 2. SELECT SP_LOGIN(\$1, \$2)

v

Capa de datos - PostgreSQL 15

| - Tablas (USUARIOS, ROLES, PERMISOS, ...)
| - Procedimientos almacenados (SP_LOGIN, SP_VALIDAR_USUARIO, ...)
| - Triggers de auditoría

| 3. Respuesta JSONB (datos + permisos)

v

Servidor: firma el token JWT y responde HTTP 200

| 4. { token, expira_en, usuario }

v

Cliente: guarda el token y lo envía en cada petición

| 5. GET /api/auth/me (Authorization: Bearer <token>)

v

Middleware autenticarToken valida la firma y expiración

5.1.1 Ventajas de la arquitectura para el proyecto

- **Rendimiento:** la lógica se ejecuta en la base de datos, evitando viajes de ida y vuelta innecesarios entre API y BD.
- **Seguridad:** la API solo ejecuta procedimientos autorizados; las claves se cifran y nunca se exponen.
- **Escalabilidad:** nuevos módulos se agregan sin reescribir la arquitectura.
- **Portabilidad:** el mismo backend sirve a aplicaciones web y móviles.
- **Auditoría:** los triggers registran automáticamente quién y cuándo modificó cada registro.

5.2 Diseño de la base de datos

El esquema relacional está conformado por siete tablas organizadas en una **jerarquía de tres niveles**: las **EMPRESAS** contienen **ESTABLECIMIENTOS** (hoteles, tiendas o restaurantes) y cada establecimiento contiene sus **AREAS** (recepción, cocina, etc.). El modelo es extensible: en el futuro se pueden agregar más empresas y cada una con sus propias cadenas.

Tabla	Descripción	Campos principales
ROLES	Roles del sistema.	NOMBRE, DESCRIPCION
EMPRESAS	Organizaciones (Aventura Vip de Cusco, Belmond, Particular).	NOMBRE, RUC, TELEFONO, DIRECCION, ESTADO

Tabla	Descripción	Campos principales
ESTABLECIMIENTOS	Sedes de una empresa (hoteles de Belmond: Monasterio, Palacio Nazarenas, Rio Sagrado).	EMPRESA_ID, NOMBRE, DESCRIPCION, ESTADO
AREAS	Sub-áreas de un establecimiento (Recepción, A&B, Cocina, Housekeeping).	ESTABLECIMIENTO_ID, NOMBRE, DESCRIPCION, ESTADO
USUARIOS	Todas las personas del sistema.	NOMBRES, APELLIDOS, DNI, CORREO, CLAVE, ROL_ID, EMPRESA_ID, AREA_ID, PUEDE_RESERVAR, CODIGO_SECRETO, ESTADO
PERMISOS	Catálogo de permisos (PBAC).	CODIGO, DESCRIPCION
USUARIOS_PERMISOS	Asignación N:M de permisos a usuarios.	USUARIO_ID, PERMISO_ID

Todas las tablas incluyen campos de auditoría: CREADO_AT, CREADO_POR, MODIFICADO_AT y MODIFICADO_POR. El trigger SP_ACTUALIZAR_MODIFICADO_AT actualiza automáticamente la fecha de modificación en cada edición.

5.2.1 Procedimientos almacenados

Siguiendo la arquitectura **SP-Centric**, el 100% de las consultas a la base de datos se ejecutan mediante procedimientos almacenados (los controladores de la API no contienen SQL directo). Los principales son:

Procedimiento	Función
SP_REGISTRAR_USUARIO	Crea usuarios cifrando la clave con bcrypt.
SP_LOGIN	Valida credenciales según el tipo de cuenta (DNI para trabajadores, correo para la app pública) y devuelve datos y permisos.

Procedimiento		Función
SP_VALIDAR_USUARIO		Verifica que un usuario siga activo.
SP_ACTUALIZAR_USUARIO		Edita datos de un usuario; cambia la clave solo si se envía una nueva.
SP_LISTAR_USUARIOS SP_OBTENER_USUARIO	/	Listado con búsqueda/filtros y detalle con permisos.
SP_CAMBIAR_ESTADO_USUARIO		Activa o inactiva (bloquea) un usuario.
SP_ASIGNAR_PERMISO SP_QUITAR_PERMISO SP_REEMPLAZAR_PERMISOS	/ /	Administración de permisos por usuario (checkboxes).
SP_GUARDAR_EMPRESA SP_GUARDAR_ESTABLECIMIENTO SP_GUARDAR_AREA	/ /	Crean o editan los catálogos de la organización.
SP_CAMBIAR_ESTADO_* (EMPRESA/ESTABLECIMIENTO/AREA)		Activa o inactiva cada entidad de la organización.
SP_LISTAR_EMPRESAS SP_LISTAR_ESTABLECIMIENTOS SP_LISTAR AREAS	/ /	Listados de gestión con conteos (establecimientos por empresa, áreas por establecimiento).
SP_OBTENER_CATALOGOS		Devuelve roles, empresas, establecimientos, áreas y permisos en un único JSONB.
SP_OBTENER_ROL_POR_NOMBRE		Devuelve el ID de un rol (valida que la operadora solo cree clientes).
SP_ACTUALIZAR_MODIFICADO_AT		Trigger: actualiza la fecha de modificación.

5.3 Diseño del módulo de autenticación

El módulo de autenticación implementa dos endpoints:

Endpoint	Método	Protegido	Función
/api/auth/login	POST	No (con rate limit)	Inicia sesión y devuelve el token JWT.
/api/auth/me	GET	Sí	Devuelve los datos del usuario autenticado.

Regla de identificador de acceso. El identificador depende del tipo de cuenta, y se valida dentro del procedimiento SP_LOGIN:

Tipo de cuenta	Identificador de acceso
Trabajadores de Aventura Vip (ADMIN, OPERADOR, CONDUCTOR)	DNI (8 dígitos)
Trabajadores de hoteles (Belmond)	DNI (8 dígitos)
App pública estilo Uber (empresa "Particular")	Correo electrónico

El login está protegido con **rate limit** (5 intentos por IP por minuto); al superarlo, la API responde HTTP 429.

5.4 Modelo de seguridad (roles y permisos)

El sistema define cuatro roles: **ADMIN**, **OPERADOR**, **CONDUCTOR** y **USUARIO**. Adicionalmente, se implementa un catálogo de **9 permisos dinámicos (PBAC)** que se asignan a cada usuario mediante checkboxes. Esto permite que existan administradores parciales (por ejemplo, un admin que solo ve finanzas, u otro que solo gestiona reservas):

Código	Descripción
GESTIONAR_USUARIOS	Crear y editar usuarios (gestión completa).
CREAR_USUARIOS	Crear y editar cuentas de clientes (rol USUARIO) - asignado a la operadora.

Código	Descripción
GESTIONAR_CATALOGOS	Gestionar empresas, establecimientos y áreas (solo admin).
VER_FINANZAS	Ver reportes financieros.
GESTIONAR_FLOTA	Gestionar la flota de vehículos.
GESTIONAR_CONDUCTORES	Gestionar conductores.
CREAR_RESERVAS	Crear reservas de viajes.
DESPACHAR_COLA	Ordenar la cola de la noche.
VER_GPS	Ver la ubicación de los vehículos en tiempo real.

Al crear un usuario, elegir su rol **pre-marca automáticamente** el conjunto de permisos recomendado (siempre editable): ADMIN recibe los 9, OPERADOR los operativos (incluido CREAR_USUARIOS), CONDUCTOR el GPS, y USUARIO ninguno (la opción de reservar se concede con el campo "Puede reservar viajes"). El menú lateral se filtra automáticamente según los permisos de la sesión.

5.5 Dashboard integral

El sistema cuenta con un **dashboard de gestión integral** con un menú lateral de navegación principal basado en iconos (lucide-react):

- **Estructura del menú:** módulos con iconos reconocibles (Inicio, Usuarios, Organización, Configuración) y soporte para submenús anidados.
- **Estados interactivos:** ítems activos (fondo ámbar + borde dorado), hover, deshabilitados sin permiso e indicadores de notificaciones (badges con contador).
- **Responsive:** en escritorio la barra lateral queda expandida; en tablet se colapsa a iconos; en móvil se convierte en drawer deslizante con menú hamburguesa.
- **Personalización:** modo oscuro/claro (persistido), accesos directos favoritos con estrella y módulos ocultos automáticamente según el rol.
- **Accesibilidad:** navegación por teclado (flechas arriba/abajo), aria-expanded, aria-current y contraste AA.
- **Área de contenido:** widgets KPI, tablas, formularios y estados de carga/error consistentes con la identidad de la marca.

5.6 Módulo de gestión de usuarios

El módulo de usuarios implementa el ciclo completo: **crear, listar, editar, activar/inactivar y administrar permisos**.

- **Listado:** tabla con búsqueda (nombre, correo o DNI), filtros por rol y estado, y acciones por fila.
- **Formulario (alta y edición):** una sola columna, agrupado por secciones (datos personales, contacto,

organización y acceso, contraseña, permisos), con validación en tiempo real, checklist de requisitos de contraseña e indicador de fortaleza.

- **Cascada empresa - establecimiento - área:** al elegir una empresa con establecimientos (como Belmond), el establecimiento y el área son obligatorios; para Aventura Vip y la app pública quedan opcionales.
- **Accesos por rol:** la operadora (permiso CREAR_USUARIOS) puede crear y editar únicamente cuentas de clientes (rol USUARIO); el admin gestiona cualquier cuenta.

5.7 Módulo de organización (catálogos de negocio)

El módulo de organización permite gestionar el catálogo de negocio de forma extensible: **empresas, establecimientos y áreas**. En Belmond los establecimientos son hoteles (Hotel Monasterio, Palacio Nazarenas, Rio Sagrado); en otra empresa podrían ser restaurantes, tiendas u otras sedes, y cada una con sus propias áreas. Solo el ADMIN gestiona estos catálogos (permiso GESTIONAR_CATALOGOS), mediante tablas con creación, edición y activación/desactivación por entidad.

CAPÍTULO VI: RESULTADOS Y ESTADO DEL DESARROLLO

6.1 Resultados obtenidos

Al cierre del presente informe se han implementado y verificado la **Fase 1 (autenticación)** y la **Fase 2 (dashboard integral, gestión de usuarios y organización)**. El proyecto cuenta con **35 pruebas de integración automatizadas** (vittest + supertest) ejecutadas contra la base de datos real; todas pasan. Entre los resultados más relevantes:

Prueba	Resultado esperado	Resultado obtenido
Login de trabajador por DNI	HTTP 200 con token	Correcto
Login de trabajador por correo	HTTP 401 (regla DNI)	Correcto
Login de app pública (Particular) por correo	HTTP 200 con token	Correcto
Clave incorrecta	HTTP 401	Correcto
Inyección SQL en el login	HTTP 401 (nunca 200)	Correcto
Rate limit del login (6º intento)	HTTP 429	Correcto
Acceso /me sin token / token manipulado	HTTP 401	Correcto

Prueba	Resultado esperado	Resultado obtenido
Listar usuarios sin permiso	HTTP 403	Correcto
Operadora crea cliente (rol USUARIO)	HTTP 201	Correcto
Operadora intenta crear ADMIN	HTTP 403	Correcto
Editar usuario sin tocar su contraseña	HTTP 200 y login sigue válido	Correcto
Bloquear usuario (INACTIVO)	Login posterior HTTP 401	Correcto
CRUD de empresas/establecimientos/áreas	HTTP 201/200	Correcto
Organización sin permiso GESTIONAR_CATALOGOS	HTTP 403	Correcto
Catálogos con la jerarquía completa	9 permisos, 3 establecimientos, 12 áreas	Correcto
PWA: manifest y Service Worker	manifest.webmanifest y sw.js generados en el build	Correcto

6.2 Estructura del proyecto desarrollado

```

avventura-vip/
+-- AGENTS.md           # Guías de trabajo para agentes IA
+-- .gitignore          # Excluye node_modules, .env y secretos
+-- back-end/
|   +-- .env / .env.example # Variables de entorno (plantilla incluida)
|   +-- package.json       # Dependencias y scripts (db:init completo)
|   +-- db/
|       +-- 00_SCHEMA.SQL   # Esquema: 7 tablas e índices
|       +-- 01_PROCEDURES.SQL # SP_LOGIN, SP_REGISTRAR_USUARIO, ...
|       +-- 02_AUDIT_TRIGGER.SQL # Triggers de auditoría

```

```

|      |-- 03_SEEDS.SQL          # Datos iniciales (9 permisos, cuentas demo)
|      |-- 05_USUARIOS_CRUD.SQL # CRUD de usuarios (SPs)
|      |-- 06_ORGANIZACION.SQL  # Empresas, establecimientos, áreas (SPs)
|      |-- 07_SP_EXTRA.SQL      # SP_OBTENER_CATALOGOS, SP_OBTENER_ROL
+-- tests/                      # 35 pruebas de integración (vitest)
+-- src/
|   |-- server.ts               # Punto de entrada
|   |-- app.ts                  # Aplicación Express (helmet, CORS, rutas)
|   |-- config/                 # database.ts y env.ts
|   |-- middlewares/           # JWT, permisos (PBAC) y errores
|   |-- controllers/           # auth, usuarios, catalogos, organizacion
|   |-- routes/                 # auth, usuarios, catalogos, organizacion
|   |-- modules/                # Tipos y contratos por módulo
+-- front-end/
+-- vite.config.ts              # Vite + PWA + proxy /api + darkMode
+-- public/                     # Logo.svg, iconos PWA y fuentes (WOFF2)
+-- src/
+-- config/                     # api.ts, navegacion.tsx, presets.ts
+-- context/                    # Tema (oscuro/claro), favoritos, navegación
+-- hooks/                      # usePermisos, useMediaQuery
+-- layouts/                     # DashboardLayout, Sidebar, Header
+-- services/                   # auth, usuarios, organizacion
+-- router/                     # Rutas protegidas por token y permisos
+-- pages/                      # Login, Inicio, Usuarios, UsuarioForm, Organizacion,
Perfil
+-- components/                 # Loader, LogoVip, CampoFormulario, ui/
+-- types/                      # Tipos de la API

```

CONCLUSIONES

1. Se implementó un módulo de autenticación seguro que valida el identificador según el tipo de cuenta (DNI para los trabajadores de Aventura Vip y de los hoteles; correo para la futura app pública), cifra las contraseñas con bcrypt, emite tokens JWT de 12 horas y protege el login con rate limit.
2. Se diseñó una base de datos relacional con una jerarquía extensible de tres niveles (empresas, establecimientos y áreas), campos de auditoría automáticos y más de 20 procedimientos almacenados: el 100% de las consultas del backend se ejecutan mediante SPs (arquitectura SP-Centric verificada).
3. El modelo de control de acceso por permisos (PBAC) con 9 permisos asignables por checkboxes permite administradores totales o parciales y define los límites de cada rol: la operadora crea y edita solo cuentas de clientes, y el menú se adapta automáticamente a los permisos.
4. Se construyó un dashboard integral responsive con menú lateral por iconos, modo oscuro/claro, favoritos y accesibilidad por teclado, que unifica la experiencia de los módulos de usuarios y organización.
5. Las 35 pruebas de integración automatizadas confirmaron el correcto funcionamiento de la autenticación, el CRUD de usuarios, los catálogos de la organización y los casos de error (401, 403, 409 y 429).

RECOMENDACIONES

1. Desarrollar la Fase 3: el módulo de reservas (tablero de la operadora, asignación de conductores, vista del cliente con GPS del vehículo y código de verificación) y el módulo de flota.
2. Desplegar el sistema en la nube (Netlify para el frontend y las funciones serverless, y una base de datos gestionada como Neon o Supabase) para que la PWA quede instalable por HTTPS desde el celular.
3. En producción, proteger la clave secreta JWT mediante un gestor de secretos y restringir los orígenes CORS al dominio oficial.
4. Realizar capacitación al personal de Aventura Vip y de los hoteles de Belmond para garantizar la adopción del sistema.

5. Documentar los nuevos módulos siguiendo la estructura y los estándares ya establecidos en las Fases 1 y 2.

REFERENCIAS BIBLIOGRÁFICAS

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. IETF.
3. Provenzano, R., & Provost, A. (2018). *Full-Stack JavaScript: A Practical Approach*. Packt Publishing.
4. Stonebraker, M. (2005). *Readings in Database Systems*. MIT Press.
5. The PostgreSQL Global Development Group. (2025). *PostgreSQL 15 Documentation*. <https://www.postgresql.org/docs/>
6. OpenJS Foundation. (2025). *Express.js Documentation*. <https://expressjs.com/>
7. React Team. (2025). *React Documentation*. <https://react.dev/>
8. Hernández Sampieri, R., Fernández Collado, C., & Baptista Lucio, P. (2014). *Metodología de la investigación*. McGraw-Hill.